

Funcionalidad: Kafka con Docker en modo KRaft

*Este documento explica en detalle **qué hace** cada componente del entorno que levantamos, por qué está configurado así, y cómo interactúan entre sí.*

1. ¿Qué es Apache Kafka?

Apache Kafka es una **plataforma de streaming de eventos distribuida**. En términos simples: es un sistema que permite que aplicaciones se comuniquen enviándose mensajes de forma confiable, escalable y persistente.

Analogía

Imagina una **cinta transportadora de fábrica**:

- Los **producers** ponen piezas (mensajes) en la cinta
- La **cinta** (Kafka) las transporta y almacena
- Los **consumers** toman las piezas cuando están listos

A diferencia de una cola de mensajes tradicional (como RabbitMQ), Kafka **no borra los mensajes** después de que son consumidos. Los guarda en disco por un tiempo configurable (por defecto 7 días), lo que permite:

- Que múltiples consumers lean el mismo mensaje
- Que un consumer “rebobine” y relea mensajes del pasado
- Auditoría y replay de eventos

Casos de uso reales

- Sistemas de pagos (Stripe, PayPal) — eventos de transacciones
 - Redes sociales — feeds en tiempo real
 - Monitoreo — métricas y logs de miles de servicios
 - E-commerce — eventos de inventario, pedidos, notificaciones
-

2. ¿Qué es KRaft y por qué reemplaza a ZooKeeper?

El problema con ZooKeeper

Hasta Kafka 2.x, el broker dependía de **Apache ZooKeeper** para:

- Guardar metadatos del clúster (qué topics existen, cuántas particiones, etc.)
- Elegir al líder de cada partición
- Coordinar los brokers entre sí

Esto significaba que tenías que mantener **dos sistemas distintos**: Kafka + ZooKeeper. Más configuración, más puntos de falla, más complejidad operacional.

KRaft: Kafka Raft Metadata Mode

KRaft (introducido en Kafka 2.8, producción desde 3.3, obligatorio en 4.x) elimina ZooKeeper y hace que **Kafka se coordine a sí mismo** usando el protocolo de consenso **Raft**.

Antes (con ZooKeeper):

ZooKeeper
(3 nodos)

coordina

Kafka KRaft
(broker + controller
en el mismo nodo)

Ahora (KRaft):

Kafka Broker

Beneficios de KRaft

- **Operación más simple**: un solo sistema para instalar y operar
- **Mayor escala**: soporte para millones de particiones por clúster
- **Inicio más rápido**: el broker no espera a ZooKeeper
- **Recuperación más rápida** ante fallos

3. Arquitectura de nuestro docker-compose.yml

```
services:
  kafka:
    image: apache/kafka:4.2.0
    container_name: kafka
    ports:
      - "9092:9092"
```

Un solo nodo combinado (broker + controller)

En nuestra configuración de desarrollo, el nodo Kafka cumple **dos roles simultáneamente**:

Container: kafka

Broker (puerto 9092)

Recibe producers/consumers

Controller (puerto 9093)

Maneja metadatos KRaft

/var/lib/kafka/data/

Mensajes persistidos en disco

localhost:9092

Tu aplicación

Esto está configurado con:

```
KAFKA_PROCESS_ROLES: broker,controller
```

En producción se separan los roles en nodos distintos para mayor resiliencia, pero para desarrollo local un nodo combinado es perfecto.

4. Explicación de cada variable de entorno

```
KAFKA_NODE_ID: 1
```

Identificador único de este nodo en el clúster. En un clúster de múltiples brokers, cada uno tendría un ID diferente (1, 2, 3...).

KAFKA_PROCESS_ROLES: broker,controller

Define qué roles juega este nodo:

- **broker**: recibe y almacena mensajes, atiende a producers y consumers
- **controller**: gestiona metadatos del clúster (topics, particiones, líderes)

KAFKA_LISTENERS

```
KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
```

Define los **sockets** en los que Kafka escucha conexiones entrantes:

- `PLAINTEXT://:9092` para producers y consumers (sin cifrado, modo desarrollo)
- `CONTROLLER://:9093` para comunicación interna del protocolo KRaft

KAFKA_ADVERTISED_LISTENERS

```
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
```

La dirección que Kafka **anuncia a los clientes** cuando se conectan. Los clients (producers/consumers) usarán esta dirección para futuras conexiones. En producción sería el hostname o IP pública del servidor.

KAFKA_CONTROLLER_QUORUM_VOTERS

```
KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
```

Lista de nodos que participan en el quórum de controllers. Formato:

`nodeId@hostname:port` . Con un solo nodo, el quórum se forma consigo mismo. En producción: `1@kafka1:9093,2@kafka2:9093,3@kafka3:9093` .

CLUSTER_ID

```
CLUSTER_ID: "MkU3OEVBNTcwNTJENDM2Qk"
```

Identificador único del clúster KRaft. Se genera una vez con `kafka-storage.sh random-uuid` y nunca cambia. Permite que los nodos verifiquen que pertenecen al mismo clúster.

KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

El topic interno `__consumer_offsets` (donde Kafka guarda el progreso de los consumers) se replica solo 1 vez. Con un broker no podemos replicar más.

5. ¿Qué es un Topic?

Un **topic** es la unidad fundamental de organización de mensajes en Kafka. Es como una **categoría** o **canal** donde los mensajes se publican y se consumen.

Topic: test-topic

Partición 0: [msg1] [msg4] [msg7]

Partición 1: [msg2] [msg5] [msg8]

Partición 2: [msg3] [msg6] [msg9]

Características:

- Un topic puede tener múltiples **particiones**
 - Los mensajes dentro de una partición están **ordenados y son inmutables**
 - Cada mensaje tiene un **offset** (número de posición) dentro de su partición
-

6. ¿Qué son las Particiones?

Las particiones son la unidad de **paralelismo y escalabilidad** en Kafka.

```
--partitions 3
```

Nuestro topic `test-topic` tiene 3 particiones. Esto significa:

Distribución de mensajes

Cuando el producer envía un mensaje **sin clave**, Kafka lo distribuye entre particiones en round-robin:

Producer envía: msg1, msg2, msg3, msg4, msg5

Partición 0: [msg1] [msg4]

Partición 1: [msg2] [msg5]

Partición 2: [msg3]

Paralelismo en consumo

Con 3 particiones, un **grupo de consumers** puede leer en paralelo con hasta 3 consumers simultáneos:

Consumer Group "mi-grupo":

Consumer A lee Partición 0

Consumer B lee Partición 1

Consumer C lee Partición 2

Si hay más consumers que particiones, los extras quedan inactivos (no reciben mensajes).

7. ¿Qué es el Replication Factor?

--replication-factor 1

El **replication factor** define cuántas copias de cada partición existen en el clúster:

Factor	Tolerancia a fallos	Requisito
1	Ninguna (dev/local)	1 broker
2	1 broker puede fallar	2 brokers
3	2 brokers pueden fallar	3 brokers

Con replication-factor 3 y 3 brokers:

Partición 0: Broker1 (líder) Broker2 (réplica) Broker3 (réplica)

Si Broker1 falla, Broker2 se convierte automáticamente en líder.

En nuestro entorno local con 1 broker, usar replication-factor 1 es la única opción.

8. Flujo completo: Producer Broker Consumer

Producer		Kafka Broker		Consumer
"Hola!"	write	test-topic	read	"Hola!"
		Partición 0: [Hola!]		
		Partición 1: []		
		Partición 2: []		

Paso a paso:

1. **Producer** conecta al broker en localhost:9092
 2. Producer envía el mensaje "Hola!" al topic test-topic
 3. Kafka determina a qué **partición** va el mensaje (round-robin sin clave)
 4. El mensaje se **escribe en disco** en el directorio /var/lib/kafka/data
 5. Se le asigna un **offset** (ej: offset 0 en partición 0)
 6. Kafka confirma al producer que el mensaje fue recibido (ack)
 7. El **consumer** hace poll al broker preguntando por mensajes nuevos
 8. Kafka entrega el mensaje al consumer
 9. El consumer **confirma** (commit) el offset leído
-

9. ¿Qué son los Offsets?

El **offset** es el número de posición de un mensaje dentro de una partición. Es como el número de página en un libro.

Partición 0:
Offset 0: "Hola Kafka!"
Offset 1: "Segundo mensaje"

Offset 2: "Tercer mensaje"

el consumer está aquí (committed offset: 2)

Kafka guarda el offset de cada consumer group en el topic interno `__consumer_offsets`. Esto permite:

- **Retomar** donde se quedó si el consumer se reinicia
 - **Múltiples grupos** independientes leyendo el mismo topic a diferentes velocidades
-

10. ¿Qué significa `--from-beginning` ?

```
kafka-console-consumer.sh --topic test-topic --from-beginning
```

Sin `--from-beginning` :

- El consumer empieza a leer **desde el final** del topic (offset más reciente)
- Solo recibe mensajes que lleguen **después** de que arrancó

Con `--from-beginning` :

- El consumer empieza desde el **offset 0** de cada partición
- Lee **todos los mensajes** históricos, luego continúa con los nuevos

Este comportamiento es controlado por la configuración `auto.offset.reset` :

- `earliest` equivale a `--from-beginning`
 - `latest` solo mensajes nuevos (comportamiento por defecto)
-

11. Persistencia de mensajes

Los mensajes en Kafka **no son volátiles**. Se almacenan en disco:

```
volumes:  
- kafka-data:/var/lib/kafka/data
```

El volumen Docker `kafka-data` persiste entre reinicios del contenedor. Si haces `docker compose down` y luego `docker compose up -d`, los mensajes siguen ahí.

Para borrar los datos: `docker compose down -v`

Retención de mensajes

Por defecto, Kafka guarda los mensajes **7 días** (`log.retention.hours=168`). Después los borra automáticamente. También puede configurarse por tamaño máximo del topic.

12. Diagrama completo de la arquitectura

```
kafka-week/infra/docker-compose.yml

Service: kafka (apache/kafka:4.2.0)

Roles: broker + controller (KRaft)

Listeners:
  PLAINTEXT    :9092 (expuesto al host)
  CONTROLLER   :9093 (interno)

Datos: /var/lib/kafka/data    volumen kafka-data

Topics creados manualmente:
  test-topic
    Partición 0  líder: broker-1
    Partición 1  líder: broker-1
    Partición 2  líder: broker-1

Host (tu máquina):
  Terminal 1: kafka-console-producer  :9092
  Terminal 2: kafka-console-consumer  :9092
```

Resumen

Concepto	Qué es	Por qué importa
KRaft	Kafka sin ZooKeeper	Operación más simple, mayor escala

Broker	Servidor que almacena mensajes	Es el corazón del sistema
Topic	Canal de mensajes	Organiza los mensajes por categoría
Partición	División del topic	Permite paralelismo y escalado
Offset	Posición de un mensaje	Permite retomar lecturas y auditar
Producer	Quien envía mensajes	Desacoplado del consumer
Consumer	Quien recibe mensajes	Lee a su propio ritmo
Consumer Group	Grupo de consumers	Distribuye particiones entre instancias
Replication Factor	Copias de cada partición	Alta disponibilidad ante fallos